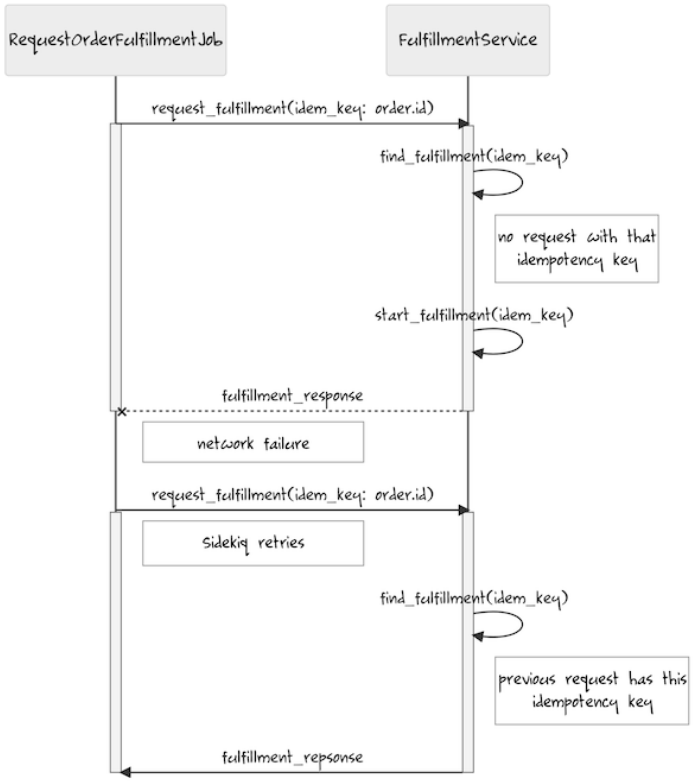


A sequence diagram showing a failed call to the fulfillment service. That call includes an idempotency key that the service remembers. When the job is retried, the same idempotency key is used, and the service locates the previous response and returns it, instead of re-executing the API call.

The trick when using idempotency keys is to choose one that truly represents a unique operation. If we always use the key “1,” then we’d never send out new orders to new customers. If we use a key based on the current time, a retried request would get a new key, and we’d send out more than one order.

In the case of the example app, no order should be fulfilled more than once. That makes the order’s primary key a good candidate for an idempotency key. Thinking through what the key should be is not always easy. Imagine if orders *could* be fulfilled more than once: the order ID would not be sufficient. You’d likely need some new concept like a `fulfillment_requests` table that tracks each fulfillment, and then use *that* table’s ID as the idempotency key.